

Activity Title

Object Classification Using Convolutional Neural Networks (Pen vs. Pencil)

Learning Objectives

At the end of this activity, students should be able to:

- 1. Understand the fundamentals of image classification using CNNs.
- 2. Prepare and preprocess an image dataset for deep learning.
- 3. Design and implement a simple CNN model.
- 4. Train, validate, and evaluate a CNN for binary classification.
- 5. Analyze model performance using accuracy and loss metrics.

Activity Description

In this activity, students will develop a Convolutional Neural Network (CNN) that can automatically classify images into two categories: **pen** and **pencil**. Students will collect or use a prepared image dataset, preprocess the images, build a CNN model, train it, and evaluate its performance.

Dataset Requirements

- Two folders:
 - pen/ – images of pens
 - pencil/ – images of pencils
- Image format: JPG or PNG
- Recommended size: **100–200 images per class**
- Images should include:
 - Different angles
 - Various backgrounds
 - Different lighting conditions

Tools and Libraries

- Python
- TensorFlow / Keras
- NumPy
- Matplotlib
- Google Colab or Jupyter Notebook

Procedure

Part 1: Data Preparation

- 1. Load the image dataset.
- 2. Resize all images to **128 × 128 pixels**.
- 3. Normalize pixel values to the range **0–1**.
- 4. Split the dataset into:
 - 80% Training
 - 20% Validation

Part 2: CNN Model Design

Students will design a CNN with the following structure:

- Input Layer: 128 × 128 × 3
- Convolution Layer + ReLU

- Max Pooling Layer
- Convolution Layer + ReLU
- Max Pooling Layer
- Flatten Layer
- Fully Connected (Dense) Layer
- Output Layer with **Sigmoid Activation** (binary classification)

Part 3: Model Training

1. Compile the model using:
 - Optimizer: Adam
 - Loss Function: Binary Crossentropy
 - Metric: Accuracy
2. Train the model for **10–15 epochs**.
3. Observe training and validation accuracy.

Part 4: Model Evaluation

1. Test the model using unseen images.
2. Record the accuracy and loss.
3. Display prediction results for sample images.
4. Identify misclassified images and possible reasons.

Expected Output

- A trained CNN model capable of classifying:
 - **Pen**
 - **Pencil**
- Accuracy graph showing training vs. validation performance.
- Sample predictions with confidence scores.

Submission Requirements

- Source code (Python notebook)
- Dataset (or link to dataset)
- Screenshots of training results
- Short reflection (3–5 sentences) on challenges encountered